

Modelos de Lenguaje y Algoritmos de Búsqueda: Análisis, Comparaciones y Diferencias

Maestría en Inteligencia Artificial · USFQ · Taller 2 Mayo 2025

Introducción

Hoy en día, cuando alguien le hace una pregunta a ChatGPT y obtiene una respuesta elaborada, o cuando busca algo en Google y recibe una lista de links relevantes, probablemente no se detiene a pensar en lo diferentes que son esos dos procesos por dentro. Y la verdad es que son bastante distintos. La intersección entre el procesamiento del lenguaje natural y la recuperación de información ha cambiado radicalmente cómo accedemos a grandes volúmenes de datos, y entender esa diferencia importa cada vez más.

Históricamente, el acceso a la información era territorio de los algoritmos de búsqueda: sistemas diseñados para indexar y recuperar documentos exactos. Pero con la llegada de los Grandes Modelos de Lenguaje (LLMs), apareció un paradigma completamente diferente: uno capaz de generar respuestas complejas y sintetizar información de formas que antes parecían imposibles. Este ensayo analiza cómo funciona por dentro la arquitectura Transformer, la compara con los algoritmos de búsqueda tradicionales, y discute si tiene sentido pensar en un LLM como un motor de búsqueda. Además, se revisa cómo la técnica RAG (Generación Aumentada por Recuperación) intenta combinar lo mejor de ambos mundos.

1. Cómo funciona por dentro la arquitectura Transformer

Para entender la relación entre los LLMs y la búsqueda de información, hay que mirar el motor que los mueve: la arquitectura Transformer. A diferencia de los modelos secuenciales anteriores, que leían el texto de izquierda a derecha, los Transformers procesan secuencias completas en paralelo gracias al mecanismo de auto-atención (Self-Attention).

En términos técnicos, un modelo Transformer no guarda un índice de textos como una base de datos. En cambio, modela la probabilidad de que ciertos tokens aparezcan juntos en un contexto dado. El mecanismo de auto-atención calcula una representación vectorial para cada palabra o fragmento, considerando el contexto de todos los demás elementos en la secuencia. Para eso, los embeddings de entrada se proyectan en tres matrices: Queries (Q), Keys (K) y Values (V). La atención se calcula mediante el producto punto entre Q y K, se normaliza con Softmax y se multiplica por V. La intuición es simple: el modelo aprende a decidir qué partes del texto son relevantes para entender cada palabra.

Durante el entrenamiento, al procesar terabytes de texto, los Transformers ajustan billones de parámetros para capturar patrones estadísticos y semánticos del lenguaje. El resultado es lo que se conoce como "conocimiento paramétrico": una representación comprimida de todo lo que el modelo ha visto. Así que cuando un LLM responde a un mensaje, predice iterativamente el siguiente token más probable dado el contexto anterior. Es un proceso probabilístico y generativo, muy distinto a hacer una consulta en una base de datos.

2. Algoritmos de búsqueda: recuperación directa y determinismo

En el otro extremo están los algoritmos de búsqueda clásicos, que funcionan de una manera mucho más directa: van al grano y devuelven exactamente lo que existe en el índice. Métodos como TF-IDF o BM25 evalúan qué tan relevante es un documento para una consulta según la frecuencia de las palabras que aparecen en él. Para funcionar, necesitan un índice invertido: una estructura de datos que mapea cada término a los documentos donde aparece. Es un enfoque directo, predecible y trazable.

Con el avance del aprendizaje profundo, los algoritmos de búsqueda dieron un salto hacia la "búsqueda semántica densa" (Dense Retrieval). En este enfoque, tanto los documentos como las consultas se convierten en vectores numéricos (embeddings), y la recuperación se basa en medir la similitud entre esos vectores, normalmente usando similitud coseno. Aunque esto ya utiliza redes neuronales similares a las de los LLMs, el objetivo no cambia: recuperar un fragmento de texto específico de un corpus fijo. La búsqueda sigue priorizando la fidelidad, la trazabilidad y la exactitud de la información.

3. ¿Los LLMs funcionan como motores de búsqueda?

La respuesta corta es: no. Los LLMs no funcionan como algoritmos de búsqueda, aunque a veces parezca que sí. La confusión es entendible: si le preguntas algo a un modelo de lenguaje y te da una respuesta con datos concretos, parece que lo está "buscando" en algún lado. Pero

por dentro, el mecanismo es completamente diferente. Hay tres razones principales por las que un LLM no puede considerarse un motor de búsqueda:

Compresión con pérdida vs. indexación exacta. Un buscador guarda los documentos de forma que pueden recuperarse tal cual, sin alteraciones. Un LLM, en cambio, actúa más como un algoritmo de compresión con pérdida: absorbe patrones del texto de entrenamiento, pero no guarda copias exactas de los documentos originales. Si le pides que reproduzca algo textualmente, puede fallar o inventar detalles.

Alucinaciones y falta de transparencia. Debido a su naturaleza probabilística, los LLMs pueden generar respuestas que suenan correctas pero que son falsas. Cuando no tienen la información necesaria o enfrentan una pregunta ambigua, tienden a "alucinar": inventan datos estadísticamente plausibles pero fácticamente incorrectos. Un motor de búsqueda simplemente devuelve un resultado vacío si el término no existe en el índice. No inventa nada.

Conocimiento congelado en el tiempo. El conocimiento de un LLM se congela en el momento en que termina su entrenamiento. Actualizarlo requiere un proceso de fine-tuning o reentrenamiento costoso computacionalmente. En cambio, agregar información nueva a un motor de búsqueda es tan sencillo como ejecutar un INSERT en la base de datos. La diferencia en agilidad es enorme.

En resumen: un buscador es un conducto directo hacia datos verificables; un LLM es un motor de inferencia y síntesis. Son herramientas distintas para propósitos distintos.

4. La convergencia: Generación Aumentada por Recuperación (RAG)

Si los LLMs generan texto fluido pero pueden inventar cosas, y los algoritmos de búsqueda son exactos pero no razonan... ¿qué pasa si los combinamos? Esa es básicamente la idea detrás de RAG (Retrieval-Augmented Generation). Esta arquitectura híbrida usa un motor de búsqueda para traer contexto relevante y verificado, y luego le pasa ese contexto al LLM para que lo procese y genere una respuesta coherente. Lo mejor de los dos mundos.

Según la literatura, RAG ha evolucionado en tres niveles de sofisticación:

Naive RAG. Es el enfoque más básico: los documentos se dividen en fragmentos (chunks), se codifican como vectores y se guardan en una base de datos vectorial. Cuando llega una consulta, el sistema recupera los fragmentos más relevantes por similitud semántica y los incluye como contexto en el prompt. El LLM genera su respuesta usando solo ese contexto. Simple, pero efectivo para reducir alucinaciones.

Advanced RAG. Agrega capas de optimización antes y después de la recuperación: mejoras en la segmentación del texto, ventanas deslizantes para capturar mejor el contexto y refinamientos en la generación de embeddings. El resultado es una recuperación más precisa.

Modular RAG. El enfoque más flexible. Permite intercambiar módulos funcionales según el contexto, y soporta recuperación iterativa o adaptativa. Ideal para flujos de trabajo dinámicos, como pipelines de Data Science o aplicaciones Full Stack que necesitan ajustarse a distintos tipos de consultas.

Con RAG, el rol del LLM cambia: deja de ser una fuente de verdad (donde puede fallar) y pasa a ser un procesador lógico de información verificada. El modelo entiende la intención del usuario, analiza lo que el motor de búsqueda recuperó y ensambla una respuesta coherente. La división de responsabilidades tiene mucho sentido.

Conclusión

La arquitectura Transformer ha transformado el procesamiento computacional del lenguaje de maneras que hace una década habrían parecido ciencia ficción. Pero una mirada técnica honesta revela que estos modelos no están diseñados para operar como motores de búsqueda, ni matemática ni arquitectónicamente. Un LLM sintetiza e interpola a partir de sus pesos paramétricos; un buscador recupera información de manera determinista a partir de índices explícitos.

Entender esa dualidad técnica lleva a una conclusión práctica: la solución no es forzar a las redes neuronales a actuar como repositorios infalibles, sino construir sistemas donde cada componente haga lo que mejor sabe hacer. Las limitaciones de los LLMs —alucinaciones, opacidad, conocimiento desactualizado— se mitigan eficazmente delegando la recuperación a verdaderos algoritmos de búsqueda, como en RAG. El futuro de la ingeniería de datos apunta hacia esos ecosistemas híbridos: motores de búsqueda que garanticen la veracidad de los datos, y modelos de lenguaje que aporten la inteligencia para procesarlos y comunicarlos de forma útil.

Referencias

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP 2020*, 6769-6781.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., & Wen, J. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1-38.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171-4186.